Foundations of Cybersecurity / Applied Cryptography

20 September 2022

Exercise n. 1 [8]

Describe the meet-in-the-middle attack to 2DES and argue about its complexity.

Exercise n. 2 [16]

Let *m* be a 256-bit message and m_i denote the *i*-th. Consider the following *one-time digital signature* scheme.

- **Key generation algorithm**.
  1. 1) Generate two random sequences S^0 and S^1, both of 256 elements, defined as follows: $S^k = \{S_i^k, 1 \leq i \leq 256\}, 0 \leq k \leq 1$, s.t., $S_i^k \leftarrow \text{random}()|_{256}$. Let $S = \{S^0, S^1\}$ be the private key.
  2. Generate two sequences P^0 and P^1, defined as follows $P^k = \{P_i^k, 1 \leq i \leq 256\}, 0 \leq k \leq 1$, s.t., $P_i^k \leftarrow H(S_i^k)$, where $H()$ is a 256-bit one-way hash function. Let $P = \{P^0, P^1\}$ be the public key.
- **Signature generation algorithm.** The digital signature *D* of *m* is defined as follows:

$$\forall 1 \leq i \leq 256, D_i = \begin{cases} S_i^0 & \text{if } m_i = 0 \\ S_i^1 & \text{if } m_i = 1 \end{cases}$$

Answer the following questions.

1) Determine the size in bit of the digital signature.
2) Specify the signature verification algorithm.
3) Argue about the unforgeability of the signature scheme.
4) Assume the secret key is used to digitally sign two different messages *x* and *y*. Let *X* and *Y* be their respective digital signatures. Describe a possible existential forgery attack that generates a valid pair (*z*, *Z*).

Exercise n. 3 [6]

Find and explain the vulnerabilities of the following function. Then patch them.

```
void ExpandVector(std::vector<int>& c) {
//This code expands vector c by doubling each element.
//For example, vector [1,2,3] becomes [1,1,2,2,3,3].
  for(auto i = c.begin(); i != c.end(); i++) {
    c.insert(i, *i);
  }
}
```

Solution

EXERCISE N.1. See theory

EXERCISE N.2

Question n.1. The digital signature size is 256 × 256 = 2^{16} = 65.536 bit.

Question n.2. The digital signature verification algorithm is:

```
bool valid = true;
for(i = 1; i ≤ 256; i++)
        if (P_i^{m_i}) ≠ H(D_i^{m_i}) {
                valid = false;
                break;
        }
```

Question n.3 Given a public key $P = \{P^0, P^1\}$, to forge the digital signature D of a message m of his choice, an adversary must determine D_i for each m_i. One possibility is to guess D_i for each message bit i. The probability of a correct guessing is $1/2^{256}$. It follows that the probability of guessing the whole digital signature is $(1/2^{256})^{256}$ which is negligible. Alternatively, the adversary may attempt to derive $D_i^{m_i}$ from $P_i^{m_i}$ but this is not computational feasible because of the one-way property of the hash function.

Question n.4. The adversary may forge a pair z and Z as follows. Initially set $z = x$ and $Z = X$. Then, since x and y are different, then there must exist at least one bit i such that $x_i \neq y_i$. For that bit set $z_i = y_i$ and $Z_i = Y_i$.
Let ν the number of bits over which x and y differ. Then it is possible to forge 2^{ν} different pairs (z, Z).

EXERCISE N.3. The function has two bugs:

1. The current version of the `insert()` operation may cause reallocation, and therefore the invalidation of iterator `i`, if the new `size()` is greater than the old `capacity()`. In this case, all iterators and references are invalidated. Otherwise, only the iterators and references before the insertion point remain valid. The *past-the-end* iterator is also invalidated."[1]
2. Furthermore, the current version of the `insert()` operation will insert a copy of the first element as many times as the iterator is incremented before getting invalid. At each iteration, to correctly specify the next element to double, iterator `i` must be incremented by two. If the iterator is incremented just by one, then il will refer to the element just inserted.

To patch the function, at each iteration, we have to assign the return value of the `insert` function to iterator `i` and then increment it by two:

```
void ExpandVector(std::vector<int>& c) {
    for(auto i = c.begin(); i != c.end(); i++) {
        i=c.insert(i, *i); // assigne the new value for the iterator
        i++ // make increment by two
    }
}
```

[1] https://en.cppreference.com/w/cpp/container/vector/insert